不正送金レビュー支援 — Databricks 実行手順

金融 / クラシックML × Databricks — エンドツーエンド・データジャーニー

ローカル版（`src/`, numpyのみ）と 1:1 対応する Databricks ネイティブ版 （`databricks/`, `app/`）の配備手順。列名・しきい値・コスト前提を ローカルと一致させてあるため、ローカルで得た数値がそのまま Databricks 上で再現できます。

0 前提

Unity Catalog 有効ワークスペース ／ Serverless SQL Warehouse ／ MLflow・Model Serving ／ Lakebase が利用可能であること。 すべて合成データを使用し、実顧客データ・PII は使用しません。

A アップロードするファイル

何を	どこへ	方法
Databricks版一式 `databricks/*, app/*`	Workspace	**GitHub に push → Databricks Repos でクローン**が最も簡単（個別 import も可）
合成生データ `data/raw_transactions.csv`	UC Volume `/Volumes/fraud_demo/fraud/raw/`	Catalog Explorer の Volume に Upload、または `databricks fs cp`

`src/`（ローカル版）とローカル生成物（`outputs/`, `logs/`, `data/lakebase.sqlite`）はアップロード不要です。

B 実行順

#	実行するもの	コンポーネント	補足
1	`databricks/02_unity_catalog_setup.sql` の**上半分**（CREATE CATALOG/SCHEMA/VOLUME＋mask関数＋GRANT）	Unity Catalog	SQLエディタで実行。GRANT先グループ（`fraud_analysts` 等）が無ければ該当行は調整/スキップ
2	`data/raw_transactions.csv` を手順1で作成した Volume にアップロード	—	`/Volumes/fraud_demo/fraud/raw/` へ
3	`databricks/01_lakeflow_ingest.py` を **Lakeflow パイプライン**のソースに指定して実行	Lakeflow	ターゲットを `fraud_demo.fraud` に設定。bronze/silver/`gold_features` が生成される
4	`databricks/03_train_model.py` を**ノートブック実行**	クラシックML	GBDT学習＋SHAP理由コード → MLflow(UC)登録 → `scored_transactions` テーブル書き出し

#	実行するもの	コンポーネント	補足
5	`02_unity_catalog_setup.sql` の `ALTER TABLE … SET MASK` 行を実行	Unity Catalog	★`scored_transactions` 生成後（=手順4の後）に実行。PIIマスク付与
6	`databricks/04_lakebase_sync.md` の手順（Lakebase インスタンス作成 → synced table → `review_decisions` テーブル）	Lakebase	Delta→Postgres 同期＋判断の書き込み先を用意
7	`databricks/05_genie_space_setup.md` の手順（Genie Space 作成＋サンプル質問6問登録）	Genie Room	対象テーブル＝`scored_transactions`
8	`app/` を `databricks apps deploy fraud-review --source-code-path .`	Databricks Apps	`app.yaml` に Lakebase 接続情報を注入。`app.py` (Flask) が起動

つまずきやすい2点（意図的に順序依存）

- **手順5のマスク付与は手順4の後。** `scored_transactions` が存在してから `ALTER TABLE … SET MASK` を実行（1回で全部流すとテーブル未作成でエラー）。
- **手順3は通常の「Run」ではなく Lakeflow（旧DLT）パイプラインとして作成**し、`01_lakeflow_ingest.py` をソース指定して起動する。

最短で「動く」を確認するなら 手順 **1 → 3 → 4** まで（Lakeflow取込 → gold → ML → スコア＋理由コード）で中核が通り、`SELECT * FROM fraud_demo.fraud.scored_transactions` が実行証跡になります。その後 6→7→8 で運用・探索・公開層を追加。

C ローカル ↔ Databricks 対応

ローカル	Databricks	一致点
`src/pipeline.py`	`01_lakeflow_ingest.py`	gold派生ロジック・列名
`src/gbdt.py`（numpy GBDT＋木経路寄与）	`03_train_model.py`（sklearn＋`shap.TreeExplainer`）	logit空間の加法的寄与＝理由コード
`src/load_lakebase.py`（sqlite）	`04_lakebase_sync.md`（Postgres synced table）	`fraud_alerts` スキーマ
`src/genie_queries.py`	`05_genie_space_setup.md`	6つの質問文と対応SQL

ローカル	Databricks	一致点
`app/local_app.py`（http.server）	`app/app.py`（Flask＋psycopg）	UI・EV計算・判断書き込み
